

Sequential Task:

Character Hierarchy

Object-Oriented Programming and GUI

March 2, 2026

1. Declare the class **Character** as the base class intended to be inherited by other character types.
 - Set its visibility according to the project requirements.
 - Design it as a foundation for polymorphism.
 - (a) Add protected fields **name** of type **String** and **strength** of type **int** so they are accessible in subclasses.
 - Ensure direct access in subclasses using the **protected** modifier.
 - Treat these fields as the core attributes of every character.
 - (b) Implement a constructor that takes **name** and **strength** and assigns them to the corresponding object fields.
 - Use the passed arguments to initialize the object state.
 - Enable proper creation of subclass instances by calling the superclass constructor.
 - (c) Define the method **performAttack()** as an extension point intended to be overridden in subclasses.
 - Set the return type to **int** representing the attack value.
 - Provide a default implementation that returns a neutral value (0), which can be replaced in subclasses.
 - Use this method as the basis for polymorphism.
 - (d) Implement the method **introduceSelf()** that returns a textual representation of the object based on **name** and **strength**.
 - Return a string containing the name and strength information in the format: **name + " (Strength: " + strength + ")"**.
 - Provide shared behavior for all subclasses.
2. Declare the class **Warrior** as a class that inherits from **Character** to extend it with a specific attack logic.
 - Use inheritance to reuse fields and methods from the base class.
 - Keep type consistency within the hierarchy.

- (a) Implement a constructor taking **name** and **strength**, and call the superclass constructor using **super**.
 - Pass the arguments to the **Character** constructor.
 - Ensure proper initialization of inherited fields.
 - (b) Override **performAttack()** to introduce custom damage calculation logic.
 - Compute damage as **(int) (strength * 1.5)**.
 - Print a message that includes the character name and dealt damage: **"Warrior " + name + " slashes with a sword dealing " + damage + " damage!"**.
 - Return the computed damage.
 - (c) Override **introduceSelf()** to extend the base behavior with character type information.
 - Add the prefix **[Warrior]** before the result from the base class.
 - Use **super.introduceSelf()** to keep the shared formatting logic.
3. This is the place where you will present your progress to the instructor.
4. Declare the class **Mage** as a class that inherits from **Character** to introduce an alternative magic-based attack logic.
- Use inheritance to reuse the fields **name** and **strength**.
 - Keep consistency with the method contracts defined in the base class.
- (a) Add a private field **rand** of type **Random** responsible for generating a random damage bonus.
 - Initialize the random generator when the object is created.
 - Restrict access to this field to within the class only.
 - (b) Implement a constructor taking **name** and **strength**, and call the superclass constructor using **super**.
 - Pass the values to the **Character** constructor.
 - Ensure proper initialization of inherited attributes.
 - (c) Override **performAttack()** to define a magic damage method.
 - Set a base magic damage value equal to 20.
 - Generate a random bonus in the range 0 to 14 using **rand.nextInt(15)**.
 - Add half of the **strength** value using integer division (**strength / 2**).
 - Print a message that includes the character name and dealt damage: **"Mage " + name + " casts a fireball dealing " + damage + " damage!"**.
 - Return the computed damage.
 - (d) Override **introduceSelf()** to extend the description with the class name.
 - Add the prefix **[Mage]** before the base class output.
 - Use **super.introduceSelf()** to keep the shared presentation logic.

5. Declare the class **Archer** as a class that inherits from **Character** to introduce a critical-hit attack mechanic.
 - Use inheritance to reuse the fields **name** and **strength**.
 - Keep compatibility with methods defined in the base class.
 - (a) Add a private field **rand** of type **Random** used to randomly determine a critical hit.
 - Initialize the random generator when the object is created.
 - Restrict access to this field to within the class only.
 - (b) Implement a constructor taking **name** and **strength**, and call the superclass constructor using **super**.
 - Pass the arguments to the **Character** constructor.
 - Ensure proper initialization of inherited attributes.
 - (c) Override **performAttack()** to implement the critical-hit logic.
 - Randomly draw a boolean value using **rand.nextBoolean()** (50% chance for critical hit).
 - Set the base damage equal to **strength**.
 - If critical hit occurs, double the damage and print: `"Archer " + name + " hits the eye! CRITICAL HIT! (" + damage + " dmg)"`.
 - Otherwise, print: `"Archer " + name + " shoots an arrow dealing " + damage + " damage."`
 - Return the final damage.
 - (d) Override **introduceSelf()** to extend the description with the class name.
 - Add the prefix **[Archer]** before the base class output.
 - Use **super.introduceSelf()** to keep the shared presentation formatting.
6. Test the created class hierarchy using the **HeroArena** class provided in this task.
7. This is the place where you will present your progress to the instructor.